Отчёт. Векторизация КТ-данных грецкого ореха: сегментация срезов и построение 3D-моделей

Студент: Леликов Слободан
3 курс, группа: БПМ-23-ПО-3
Проверил: доцент, Полевой Д. В.
Github: ssldeaf/misis2026s-23-03-lelikov-s-s

Содержание

1. Постановка задачи.....3
2. Данные и целевые классы.....3
3. Сегментация: архитектура пайплайна (segm.py).....6
3.1. Стадия A: маска ореха (build_walnut_mask).....7
3.2. Стадия B: скорлупа и внутренность (build_shell).....10
3.3. Стадия C: ядро (build_kernel).....13
3.4. Стадия D: перегородки (build_partitions).....17
3.5. Итоговый overlay среза.....19
4. Оценка точности сегментации.....20
4.1. Метрики Dice и IoU.....20
4.2. Контроль площадей по объёму (area_check.py).....21
5. Построение 3D-мешей (build_mesh.py).....21
5.1. Декодирование overlay в метки (decode_labels).....22
5.2. Базовый метод и первые артефакты.....22
5.3. Отладка: очистка и устранение артефактов.....23
5.4. Итоговые меши.....26
6. Результаты.....27
7. Структура репозитория.....28

1. Постановка задачи

Дан КТ-скан грецкого ореха (датасет Walnut2), состоящий из 501 среза в формате TIFF. Конечная цель работы — получить связное трёхмерное представление ореха, разделённое по анатомическим классам: скорлупа, ядро и перегородки. Работа разбита на два больших этапа, выполняемых последовательно.

Этап 1 — сегментация (segm.py)

Вход этапа — отдельный КТ-срез (8-битное изображение после нормализации). Выход — для каждого среза три бинарные маски (скорлупа, ядро, перегородки) и контрольный цветной overlay. Этап решает задачу классификации каждого пикселя среза в один из четырёх классов (фон / скорлупа / ядро / перегородка) на основе яркости, формы и взаимного расположения структур. Алгоритм работает посрезово и параметрически: числовые пороги, чувствительные к положению среза вдоль ореха, интерполируются по номеру среза (si). Этап реализован в segm.py и состоит из четырёх последовательных стадий $A \to B \to C \to D$ (раздел 3), где маска каждой стадии ограничивает область поиска следующей. Результат всех срезов — стопка overlay-изображений, которая служит входом второго этапа.

Этап 2 — векторизация и построение 3D-мешей (build_mesh.py)

Вход этапа — стопка overlay-изображений (результат этапа 1), сложенная в трёхмерный воксельный массив. Выход — три полигональные поверхности (mesh) классов в форматах OBJ и PLY. Этап решает задачу восстановления непрерывной 3D-поверхности из дискретного набора 2D-масок: цвета overlay декодируются обратно в метки классов, по каждому классу строится изоповерхность методом Marching Cubes, после чего поверхность очищается от артефактов (мелкие компоненты, обрезанные края, паразитные заполнения) и приводится к виду, пригодному для дальнейшей работы с сетками. Этап реализован в build_mesh.py (раздел 5).

2. Данные и целевые классы

Исходные данные — стопка из 501 среза формата TIFF (16-битные значения плотности рентгеновского поглощения). Крайние срезы, на которых орех отсутствует или представлен единичными пикселями, из обработки исключены: рабочий диапазон — срезы с 69 по 431 (около 372 срезов).

Прежде чем переходить к алгоритму, зафиксируем, что именно требуется получить. Ниже на одном срезе (si = 303) показаны исходное изображение ореха и три целевые маски-класса, которые должен выдать этап сегментации.

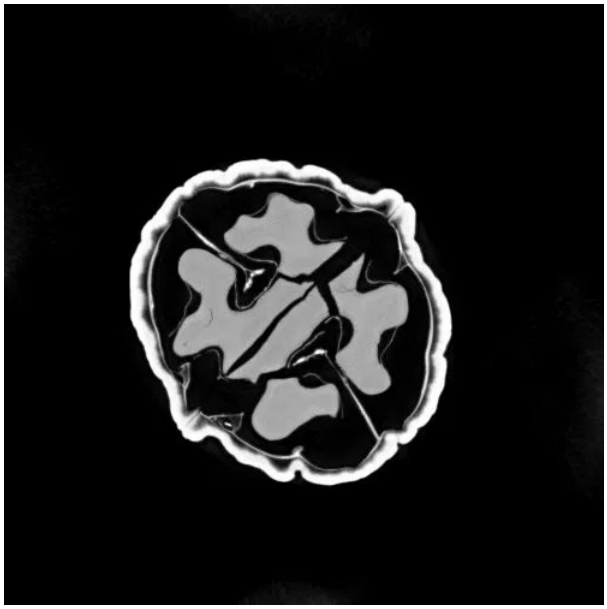


Рис. 1. Исходный КТ-срез ореха (si = 303): яркая скорлупа-кольцо, тёмные полости, ядро и тонкие перегородки.

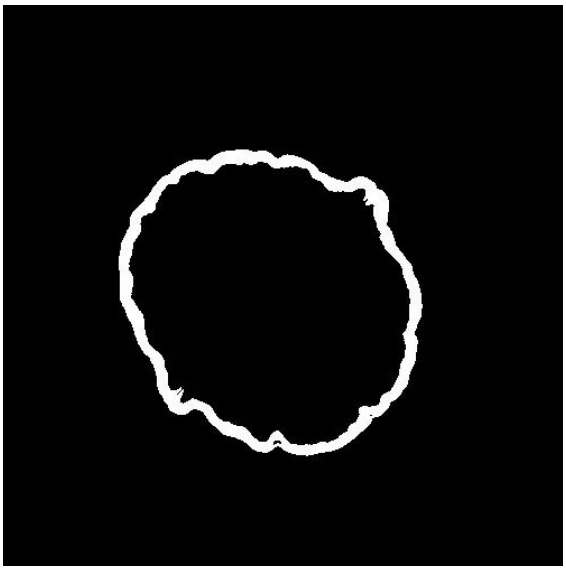


Рис. 2. Целевая маска класса «скорлупа» (shell) — внешняя оболочка ореха.

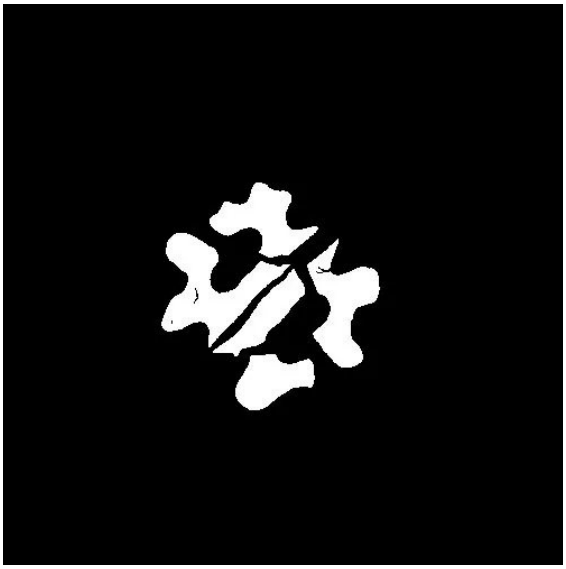


Рис. 3. Целевая маска класса «ядро» (kernel) — лопасти съедобной части.

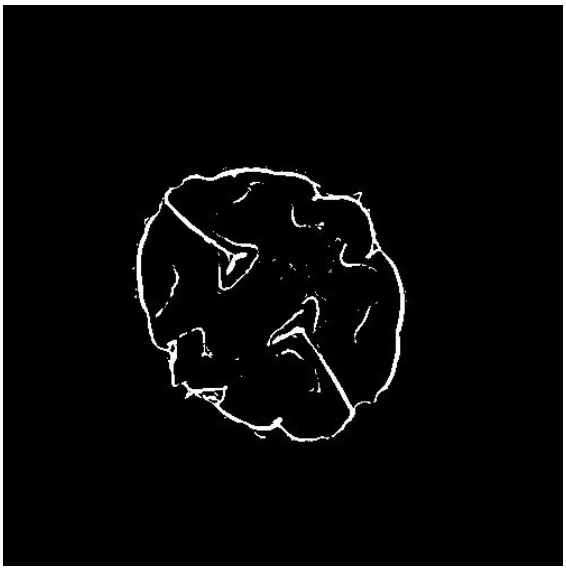


Рис. 4. Целевая маска класса «перегородки» (septa) — тонкие перегородки между лопастями.

Задача этапа 1 — построить эти три маски автоматически для каждого среза. Ниже описано, как load_slice готовит изображение, после чего пайплайн стадий A–D последовательно выделяет классы.

Загрузка и нормализация одного среза выполняется функцией load_slice(path):

```
def load_slice(path):
    raw = tifffile.imread(path).astype(np.float32)
    p_low  = np.percentile(raw, 1)      # перцентильное
    p_high = np.percentile(raw, 99)     # клиппирование
    raw = np.clip(raw, p_low, p_high)   # убираем выбросы (топ/низ 1%)
    img = cv2.normalize(raw, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
    # si извлекается из имени файла регулярным выражением
    return img, img, si
```

Клиппирование по 1-му и 99-му перцентилям отсекает единичные сверхъяркие/сверхтёмные пиксели, которые иначе сжали бы полезный диапазон яркостей ореха при нормализации. Результат — 8-битное изображение, готовое к сегментации (Рис. 5).

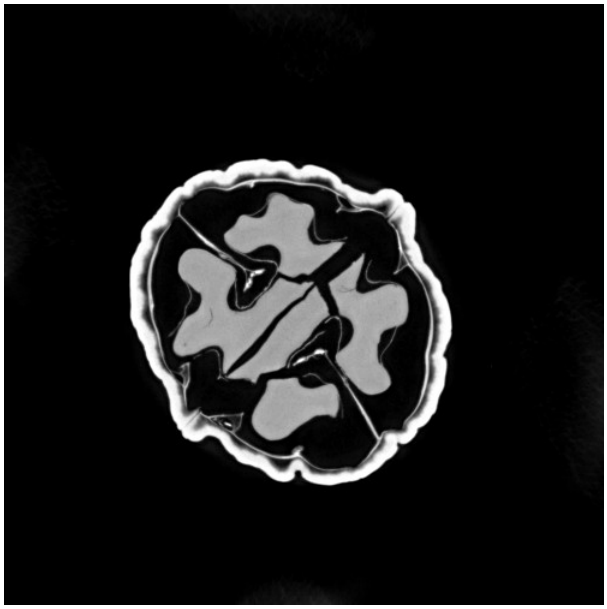


Рис. 5. Срез si = 303 после загрузки, клиппирования и нормализации в диапазон 0–255.

3. Сегментация: архитектура пайплайна (segm.py)

Пайплайн разбит на четыре последовательные стадии. Маски, найденные на предыдущей стадии, используются как ограничители или подсказки для следующей:

— **A** `build_walnut_mask` — построение маски всего ореха;

— **B** `build_shell` — отделение скорлупы от внутренней части (interior);

— **C** `build_kernel` — выделение ядра внутри interior;

— **D** `build_partitions` — выделение перегородок между лопастями ядра и в зоне скорлупы.

Параметризация по срезу

Все числовые параметры, чувствительные к номеру среза, вынесены в словарь PARAM_ANCHORS и интерполируются функцией interp_param(name, si). Словарь хранит значения в виде пар (si, value); функция сортирует анкоры и возвращает линейно проинтерполированное значение между двумя соседними. Если si меньше минимального или больше максимального анкора — берётся значение крайнего.

```
def interp_param(name, si):
    anchors = sorted(PARAM_ANCHORS[name])
    if si <= anchors[0][0]:  return anchors[0][1]
    if si >= anchors[-1][0]: return anchors[-1][1]
    for (xa, pa), (xb, pb) in zip(anchors[:-1], anchors[1:]):
        if xa <= si <= xb:
            return pa + (pb - pa) * (si - xa) / (xb - xa)
```

Такой подход позволяет точечно настраивать алгоритм для отдельных диапазонов срезов, не трогая логику. Структурные параметры, не зависящие от si (размеры морфологических ядер, пороги площадей), оставлены отдельными константами.

Debug-механизм

Функция process_slice(path, debug, output_dir) оркеструет все стадии. При debug=True каждой стадии передаётся коллбэк save(name, image), который сохраняет

промежуточный результат под именем NNN_func__step_siXXX.png (NNN — сквозной счётчик вызовов). Так каждая морфологическая операция фиксируется отдельным изображением, а их порядок воспроизводим. Команда запуска:

```
python segm.py --mode debug      # один срез, со всеми промежуточными шагами
```

Прогон на срезе 303 дал 48 пронумерованных изображений. Ниже показаны ключевые из них по каждой стадии.

3.1. Стадия A: маска ореха (build_walnut_mask)

Функция строит бинарную маску всего ореха. Ниже каждый шаг показан и обоснован — в том числе наложением на исходное изображение и сравнением с эталоном (числами).

Шаг 1 — порог OTSU и зачем он нужен

Скорлупа на КТ заметно ярче фона и большей части внутренних полостей. Метод OTSU (cv2.THRESH_OTSU) автоматически подбирает порог, максимизирующий межклассовую дисперсию яркостей — то есть сам находит границу «тёмное/светлое» без ручного задания значения. Результат бинаризации показан на Рис. 6, а на Рис. 7 он наложен зелёным на исходный срез: видно, что OTSU уверенно захватывает яркое кольцо скорлупы и светлые фрагменты, но оставляет тёмные внутренние полости незаполненными (это и есть причина последующих шагов).

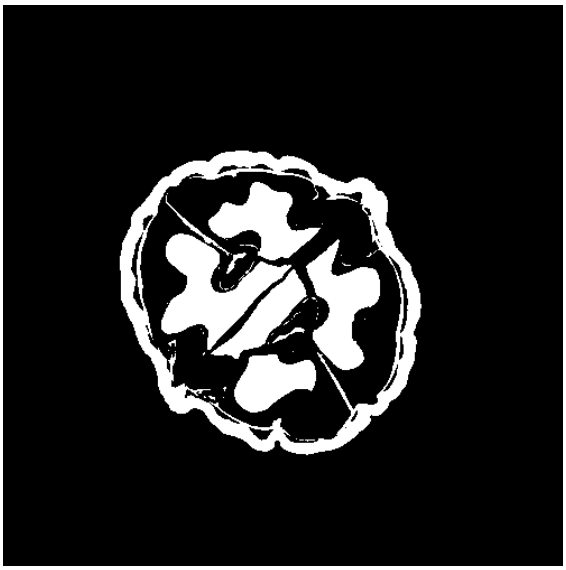


Рис. 6. Стадия A, шаг 1: бинаризация методом OTSU.

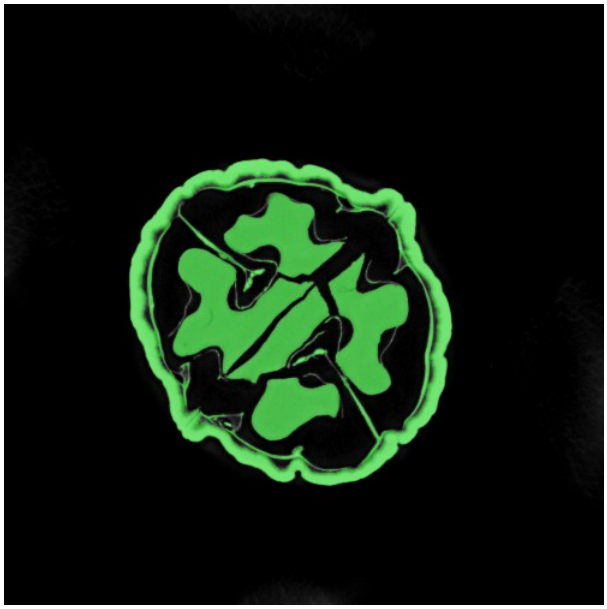


Рис. 7. Полезность OTSU: результат порога (зелёный) наложен на исходный срез. Захвачено 28 781 px — яркая скорлупа и светлые участки; тёмные полости внутри пока вне маски.

Шаг 2 — морфологическое закрытие и заливка контура

Сырой результат OTSU содержит крупные «дыры»: тёмные полости и перегородки внутри ореха порог не захватил. Но для маски ореха нам нужен сплошной силуэт. Морфологическое закрытие ядром 7×7 (дилатация, затем эрозия) смыкает разрывы, после чего заливка наибольшего контура (шаг 3) закрывает оставшиеся внутренние полости целиком. На Рис. 8 показан результат закрытия, а на Рис. 9 — наглядно, что именно добавляют закрытие и заливка: зелёным исходный OTSU, жёлтым — восстановленные внутренние области.

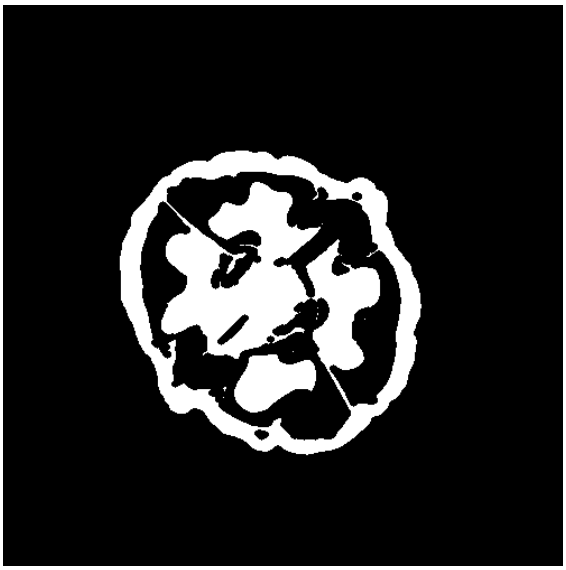


Рис. 8. Стадия А, шаг 2: результат морфологического закрытия.

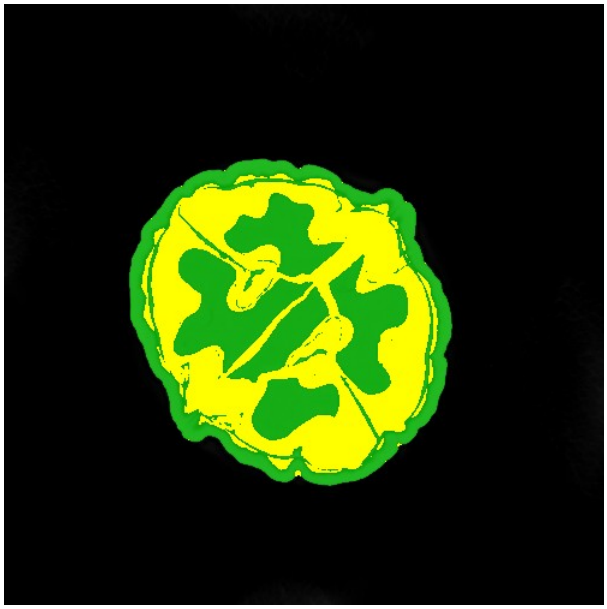


Рис. 9. Смысл морфологии: зелёным — что нашёл OTSU (28 781 px), жёлтым — внутренние полости (≈ 27 100 px), восстановленные закрытием и заливкой контура. Без этого шага маска ореха была бы «дырявой».

Шаг 3 — выбор наибольшего центрального контура

Среди всех внешних контуров (cv2.findContours) отбираются прошедшие два фильтра: площадь не меньше min_area = 5000 px (отсекает мелкий шум) и центр масс не дальше 0.4·min(H,W) от центра кадра (отсекает посторонние объекты у краёв). Из прошедших выбирается наибольший и заливается сплошным белым (Рис. 10). Эта залитая маска уже совпадает с эталонным силуэтом ореха на Dice = 0.982 (IoU = 0.964).

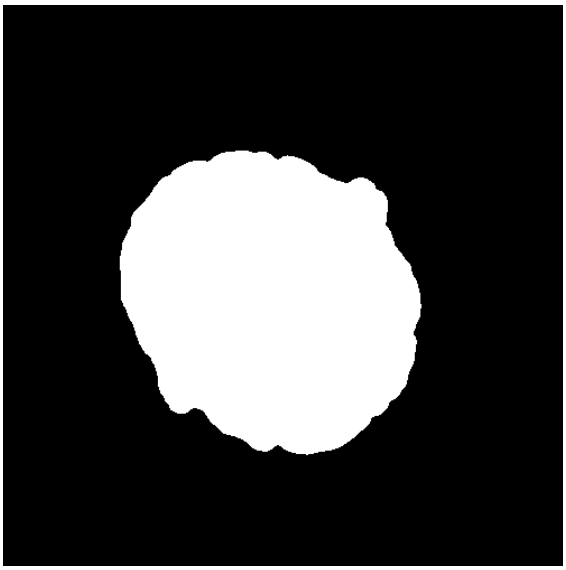


Рис. 10. Стадия A, шаг 3: выбран и залит наибольший центральный контур.

Шаг 4 — дилатация и контроль по эталону

Заливка контура немного «недотягивает» до внешней кромки скорлупы. Лёгкая дилатация ядром 3×3 расширяет маску на ~1 пиксель наружу, гарантированно захватывая край скорлупы. Эффект небольшой по площади (+806 px), но повышает согласие с эталоном: Dice растёт с 0.982 до 0.989 (IoU 0.964 → 0.978). На Рис. 11 красным показано кольцо, добавленное дилатацией, на Рис. 12 — сравнение итоговой маски с

эталоном (зелёный — совпадение, синий — недосегментация 1267 px, красный — лишнее, всего 8 px).

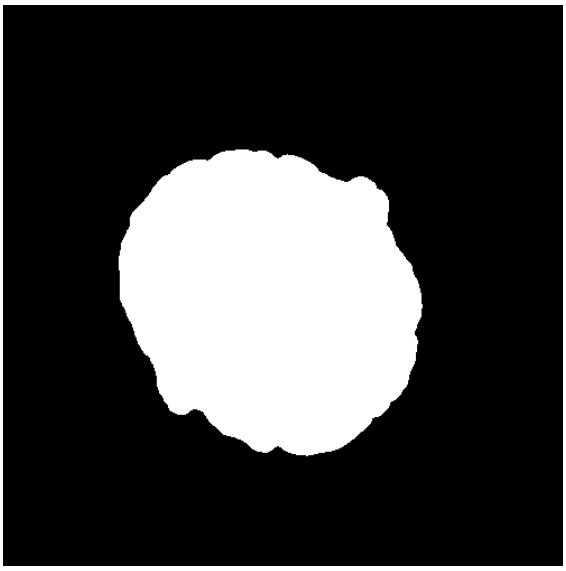
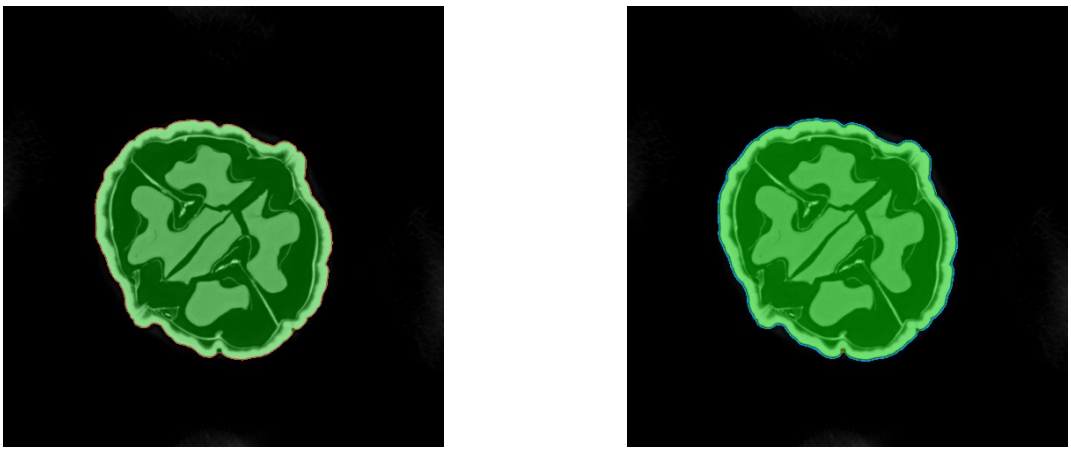


Рис. 11. Стадия A, шаг 4: итоговая маска ореха.



красное кольцо — добавлено дилатацией (+806 px) *сравнение с эталоном: TP (зел.), FN (син.), FP (красн.)*

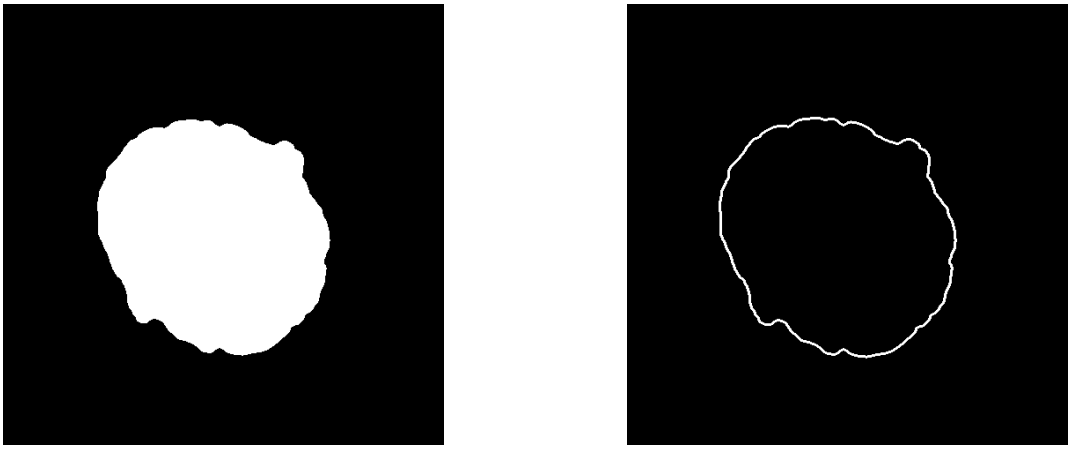
Рис. 12. Полезность дилатации и контроль по эталону. Итоговая маска ореха: Dice = 0.989, IoU = 0.978 относительно эталонного силуэта.

3.2. Стадия B: скорлупа и внутренность (build_shell)

Функция build_shell(bl, mask, si) разделяет маску ореха на скорлупу (shell) и внутренность (interior).

Определение interior на этой стадии: interior — это всё, что внутри ореха за вычетом скорлупы, то есть (mask − shell), жёстко ограниченное внешним контуром скорлупы. Геометрически это «начинка» ореха: полости, ядро и перегородки вместе. Именно этот interior передаётся дальше в стадии C и E как область поиска. (В этих стадиях он используется по-разному — см. соответствующие разделы.)

1) Маска ореха эродируется диском 7×7 — получается суженная область me. Разность (mask − me) даёт тонкое внешнее кольцо outer, которое всегда относится к скорлупе (Рис. 13).



эрозия (me) *кольцо outer = mask − me*

Рис. 13. Стадия B: эрозия маски и выделение внешнего кольца скорлупы.

2) Внутри me берутся пиксели исходной яркости; вычисляется перцентильный порог t (для среза 303: p = 85 → t = 176): скорлупа ярче начинки, поэтому верхний перцентиль яркости внутри области me соответствует внутренней стенке скорлупы. Бинаризация по этому порогу — Рис. 14.

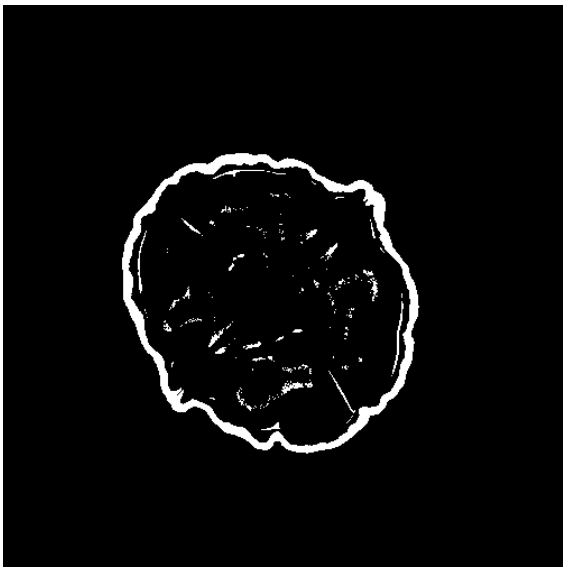
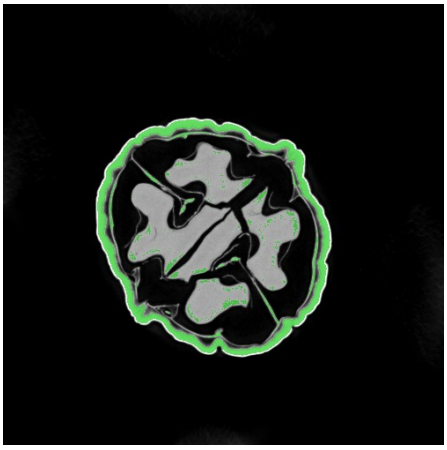


Рис. 14. Стадия B, шаг 2: бинаризация области me по перцентильному порогу (p = 85, t = 176).

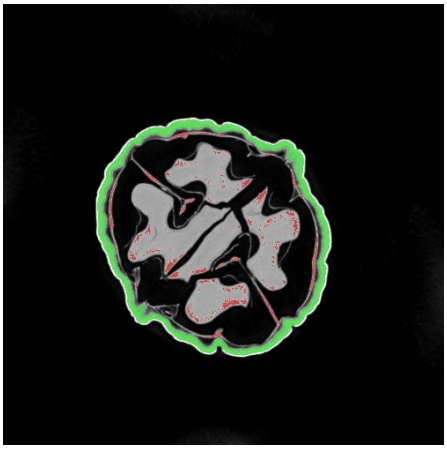
Шаг 3 — отбор компонент: зелёным берём, красным выбрасываем

После порога остаётся набор связных компонент. Их нужно отфильтровать: к скорлупе относится одна крупная кольцевая компонента (внутренняя стенка), а множество мелких ярких пятен внутри начинки — это блики на ядре и перегородках, которые надо отбросить. Алгоритм для срезов si > 70 принимает только наибольшую по площади компоненту, остальные отклоняет.

На отладочных кадрах это показано цветом на исходном срезе: зелёным закрашены принятые пиксели, красным — отклонённые. На Рис. 15 (слева) показаны все найденные компоненты до фильтра — зелёное кольцо стенки плюс множество красноватых вкраплений внутри (яркие точки на ядре). На Рис. 15 (справа) — после фильтра: оставлена только кольцевая компонента, все внутренние пятна отброшены (красным). Принятая компонента объединяется с кольцом outer — это и есть маска скорлупы.



все компоненты (зелёное — кандидаты)



после фильтра: зелёное — принято, красное — отброшено

Рис. 15. Стадия B, шаг 3: отбор компонент скорлупы. Принимается одна крупная кольцевая компонента (зелёная), внутренние яркие пятна отбрасываются (красные).

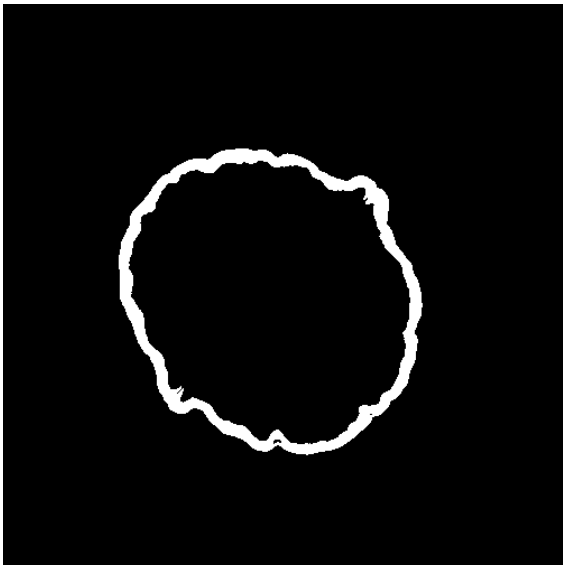


Рис. 16. Стадия B: итоговая маска скорлупы (кольцевая компонента + внешнее кольцо outer).

4) Внутренность вычисляется как (mask − shell), ограниченная внешним контуром скорлупы. Итог стадии — пара масок (shell, interior); interior показан на Рис. 17.

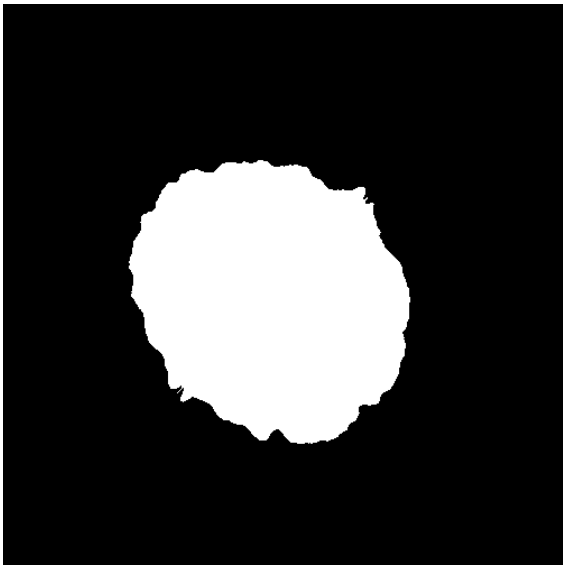


Рис. 17. Стадия B: маска внутренней области ореха (interior = mask − shell).

3.3. Стадия C: ядро (build_kernel)

Определение interior на этой стадии: здесь interior — это область поиска ядра, то есть вся начинка ореха (полученная в стадии B). Region growing запускается только внутри interior, и любой результат жёстко обрезается по нему: ядро по определению не может выйти за внутреннюю стенку скорлупы. Таким образом interior выступает «маской-ограничителем».

Функция build_kernel(bl, interior, si) активна только в диапазоне срезов [84, 383]; вне его возвращается пустая маска. Из interp_param читаются параметры: seed_pct (порог яркости для «семян»), solidity_min и aspect_min (минимальные плотность выпуклой оболочки и вытянутость).

region_growing — выращивание области

Алгоритм на основе 4-связности. Яркие пиксели interior (выше процентильного порога seed_pct) берутся как зёрна; от каждого зерна область растёт на соседей, чья яркость отличается от текущего среднего региона не более чем на TOLERANCE. Процесс идёт, пока есть подходящие соседи. 4-связность (вместо 8-) выбрана из-за биологической природы структуры: диагональные «перетекания» между лопастями нежелательны. Результат обрезается по interior (Рис. 18).

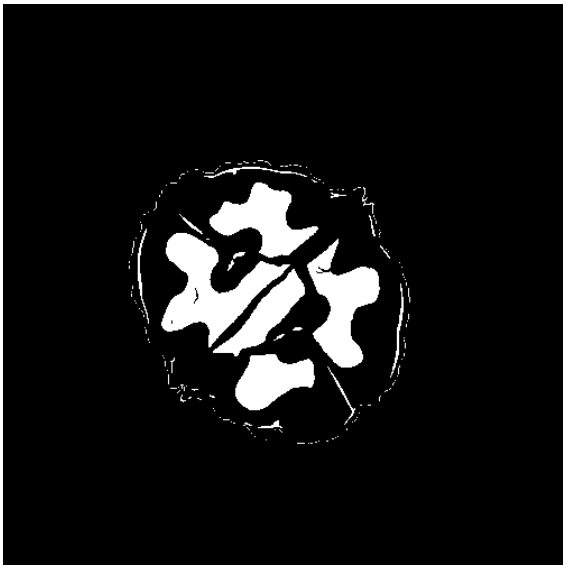


Рис. 18. Стадия C: результат region_growing (сырое ядро до фильтрации формы).

Контроль формы: последовательность фильтров (зелёным берём, красным выбрасываем)

Сырой результат содержит как настоящие лопасти ядра, так и ложные компоненты (мелкие пятна, тонкие затёки, рыхлые области). Они отсеиваются цепочкой из четырёх геометрических фильтров; на каждом отладочном кадре принятые компоненты показаны зелёным, отклонённые — красным, рядом подписана площадь a каждой компоненты.

Фильтр 1 — площадь. Компоненты меньше min_lobe = max(50, int_area × area_frac) ≈ 716 px отбрасываются как шум. На Рис. 19 крупные лопасти зелёные, мелкие точки по периметру — красные.



Рис. 19. Стадия C, фильтр 1 (площадь ≥ 716 px): крупные лопасти приняты (зелёные), мелкие компоненты отклонены (красные).

Фильтр 2 — Solidity (плотность). Solidity = площадь / площадь выпуклой оболочки. Настоящая лопасть плотная и выпуклая (Solidity высокая); рыхлые, изрезанные компоненты с Solidity < solidity_min ≈ 0.36 отбрасываются (Рис. 20).



Рис. 20. Стадия C, фильтр 2 (Solidity ≥ 0.36): рыхлые/изрезанные компоненты отклоняются.

Фильтр 3 — Aspect (вытянутость). Aspect = малая ось / большая ось эллипса. Лопасть округлая (Aspect не слишком мал); тонкие иглоподобные компоненты с Aspect < aspect_min ≈ 0.15 отбрасываются (Рис. 21).



Рис. 21. Стадия C, фильтр 3 (Aspect ≥ 0.15): тонкие иглоподобные компоненты отклоняются.

Фильтр 4 — доля дыр. Если внутренние дыры занимают больше 20 % площади компоненты, она бракуется как ненадёжная (Рис. 22). Комбинированный результат всех четырёх фильтров — Рис. 23.



Рис. 22. Стадия C, фильтр 4 (доля дыр ≤ 20 %): компоненты с большими внутренними полостями отклоняются.



Рис. 23. Стадия C: комбинированный результат фильтрации формы (приняты только настоящие лопасти).

Итог стадии — маска ядра (Рис. 24).

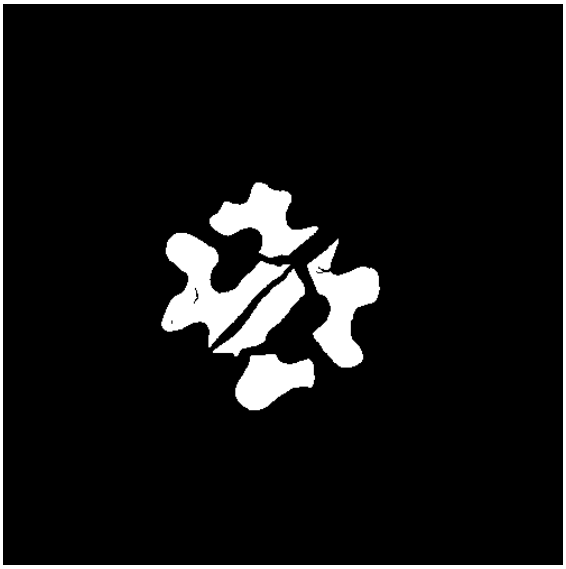


Рис. 24. Стадия C: итоговая маска ядра.

3.4. Стадия D: перегородки (build_partitions)

Определение interior на этой стадии: здесь используется не сам interior, а его эродированная версия interior_eroded = erode(interior, 3px). Смысл сужения — исключить краевую полосу шириной 3 пикселя у внутренней стенки скорлупы: на этой границе Frangi-фильтр даёт ложные срабатывания (резкий перепад яркости стенки похож на линейную структуру). Таким образом на этой стадии «рабочий interior» намеренно уже, чем на стадиях B и C.

Перегородка — это тёмная линейная полоска. Перегородки в орехе неоднородны: одни проходят через центр interior, другие примыкают к ядру, третьи — к внутренней стенке скорлупы. Один метод все типы не ловит, поэтому build_partitions объединяет три независимых источника.

Источник 1 — Frangi-фильтр

Маска interior эродируется на 3 пикселя (исключаем краевую полосу у скорлупы, где Frangi даёт ложные срабатывания). К изображению, обнулённому вне этой зоны, применяется Frangi-фильтр (black_ridges=True, sigmas = 1..3). Фильтр анализирует собственные значения матрицы Гессе: вдоль тёмного гребня кривизна почти нулевая ($\lambda_1 \approx 0$), поперёк — большая ($\lambda_2 \gg 0$); такой точке присваивается высокий отклик (Рис. 25).

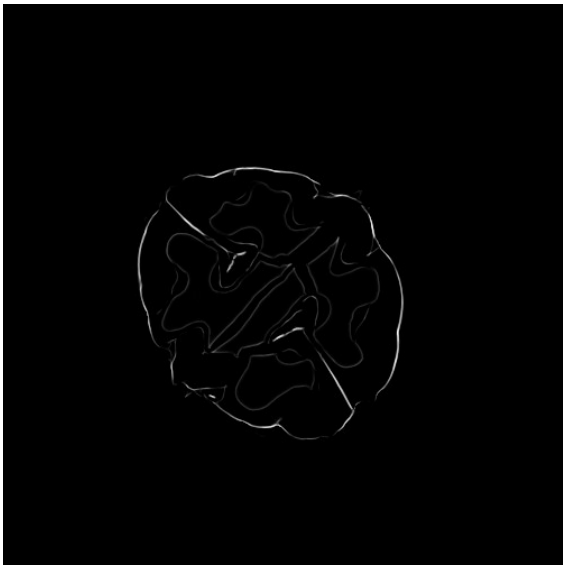


Рис. 25. Стадия D: отклик Frangi-фильтра (линейные тёмные структуры).

Отклик нормализуется и порогуется (оставляем верхние проценты по величине). Затем из результата вычитается расширенная маска ядра — убираем пиксели на его контуре, которые Frangi принял за перегородки из-за резкого перепада яркости на границе (Рис. 26).

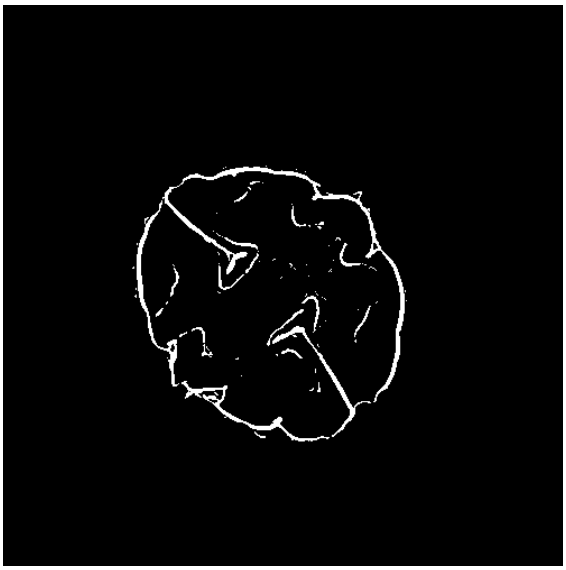


Рис. 26. Стадия D: источник 1 после порогования и удаления зоны ядра.

Источник 2 — кольцо вокруг ядра (ring bridge)

Вокруг ядра строится кольцо шириной RING_PX = 20 пикселей — именно здесь перегородки отходят от лопастей. Внутри кольца берутся пиксели ярче порога ring_bright_pct, и оставляются только компоненты, касающиеся ядра. Это «мост» — перегородка, начинающаяся прямо у края лопасти (Рис. 27).

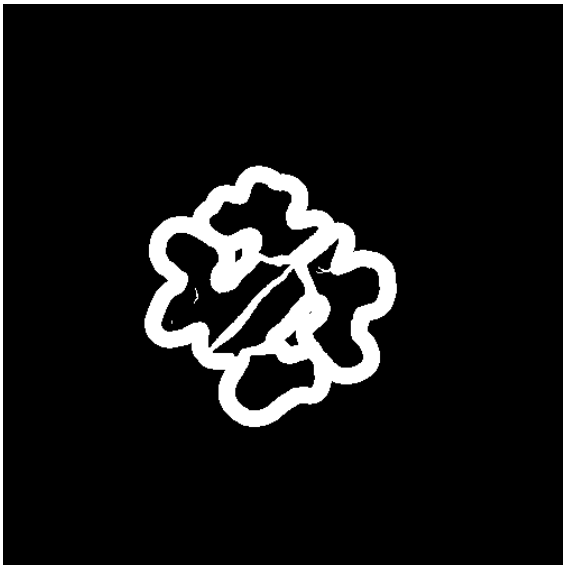
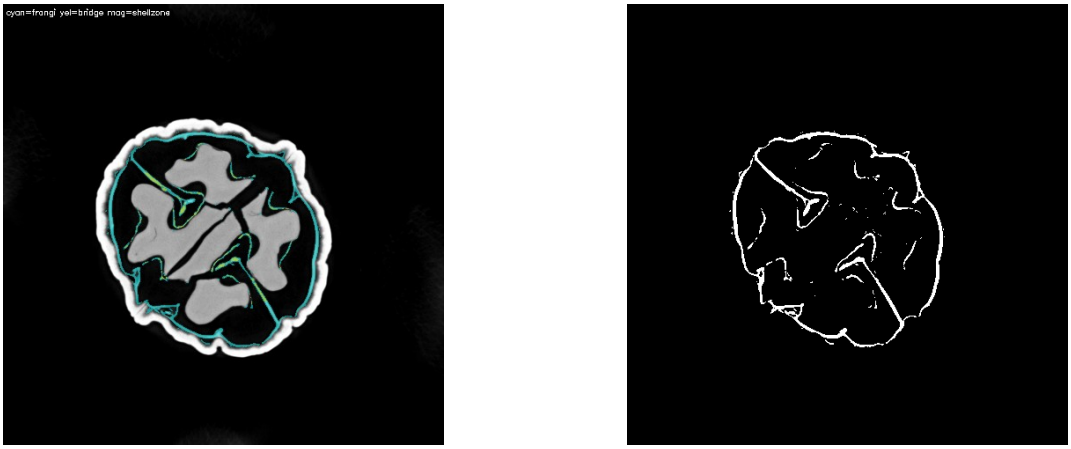


Рис. 27. Стадия D: источник 2 — кольцевая зона вокруг ядра.

Источник 3 — зона скорлупы (shell zone)

Зона между interior и скорлупой: здесь тоже бывают перегородки, примыкающие к внутренней стенке. Из неё берутся пиксели ярче фиксированного порога с фильтром по площади.

Три источника объединяются через bitwise_or (берётся всё, что хотя бы один детектор счёл перегородкой), результат обрезается по маске ореха, из него удаляется скорлупа. Цветная визуализация вкладов трёх источников и итоговая маска перегородок — Рис. 28.



вклад трёх источников (цветом) *итоговая маска перегородок*

Рис. 28. Стадия D: объединение трёх источников и итоговая маска перегородок.

3.5. Итоговый overlay среза

Функция make_overlay(r) собирает цветной контрольный overlay: серое изображение переводится в BGR, поверх него полупрозрачно (cv2.addWeighted) накладываются маски — скорлупа зелёным, ядро красным, перегородки жёлтым. Итог для среза 303 — Рис. 29.

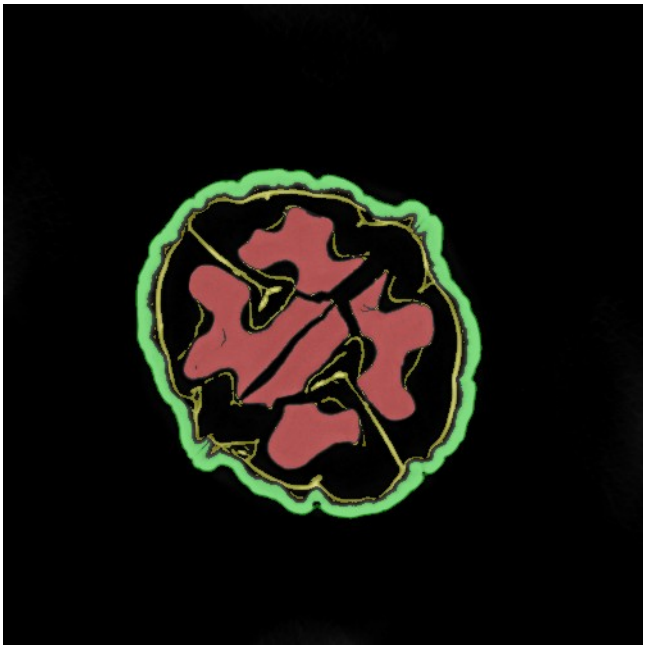


Рис. 29. Итоговый overlay среза si = 303: скорлупа (зелёный), ядро (красный), перегородки (жёлтый).

4. Оценка точности сегментации

Качество сегментации оценивалось двумя независимыми способами: метриками перекрытия с эталонными масками (Dice, IoU) и контролем плавности площадей классов по объёму.

4.1. Метрики Dice и IoU

Подготовлены 42 эталонные маски (по 14 на класс) — слайсы с 123 по 383 с шагом 20, равномерно по всему объёму. Для тех же срезов взяты маски, выданные алгоритмом. Сравнение выполнено скриптом evaluate.py, который для каждой пары считает:

```
dice = 2 * |A ∩ B| / (|A| + |B|)
iou  = |A ∩ B| / |A ∪ B|
```

Эталоны хранятся в палитровом TIFF с инвертированной палитрой (индекс 0 = белый фон, объект — тёмный), что evaluate.py корректно учитывает через ColorMap. Срез 383 для ядра исключён из статистики: на нём ядра уже нет, и его включение исказило бы среднее. Сводные результаты — в таблице ниже (получены реальным прогоном python evaluate.py).

Класс	Dice (mean ± std)	IoU (mean ± std)	Срезов
Ядро (kernel)	0.929 ± 0.016	0.869 ± 0.028	13
Скорлупа (shell)	0.885 ± 0.010	0.795 ± 0.017	14
Перегородки (septa)	0.773 ± 0.034	0.631 ± 0.044	14
Всего (41 маска)	**0.860 ± 0.070**	**0.761 ± 0.105**	**41**

Таблица 1. Метрики перекрытия с эталонными масками по классам.

Средний Dice 0.860 по всем структурам свидетельствует о хорошей точности автоматической сегментации. Наилучшее качество достигнуто для ядра (Dice 0.929) — оно крупное, плотное и контрастное. Скорлупа сегментируется стабильно (Dice 0.885, наименьший разброс). Перегородки ожидаемо сложнее всего (Dice 0.773): это тонкие

структуры, где даже сдвиг на 1–2 пикселя сильно бьёт по метрике перекрытия. Высокий суммарный разброс (std 0.070 по Dice) объясняется не нестабильностью алгоритма, а принципиальной разницей структур: ядро и перегородки — объекты разной геометрической природы.

4.2. Контроль площадей по объёму (area_check.py)

Вторая, независимая от эталонов проверка опирается на физическую непрерывность: площадь каждого класса меняется вдоль оси z плавно. Резкий скачок площади на отдельном срезе — признак ошибки сегментации или артефакта. Скрипт area_check.py прогоняет пайплайн по всем срезам, собирает площадь каждого класса и строит для каждого baseline скользящей медианой (окно WINDOW = 15); срезы, где площадь отклоняется от baseline более чем на THRESHOLD (35 %), помечаются как аномальные.

График ниже (Рис. 30) построен по всем срезам рабочего объёма. Кривые площадей трёх классов гладкие, что подтверждает объёмную связность сегментации; единичные помеченные точки соответствуют переходным срезам на границах присутствия класса, а не систематическим ошибкам.

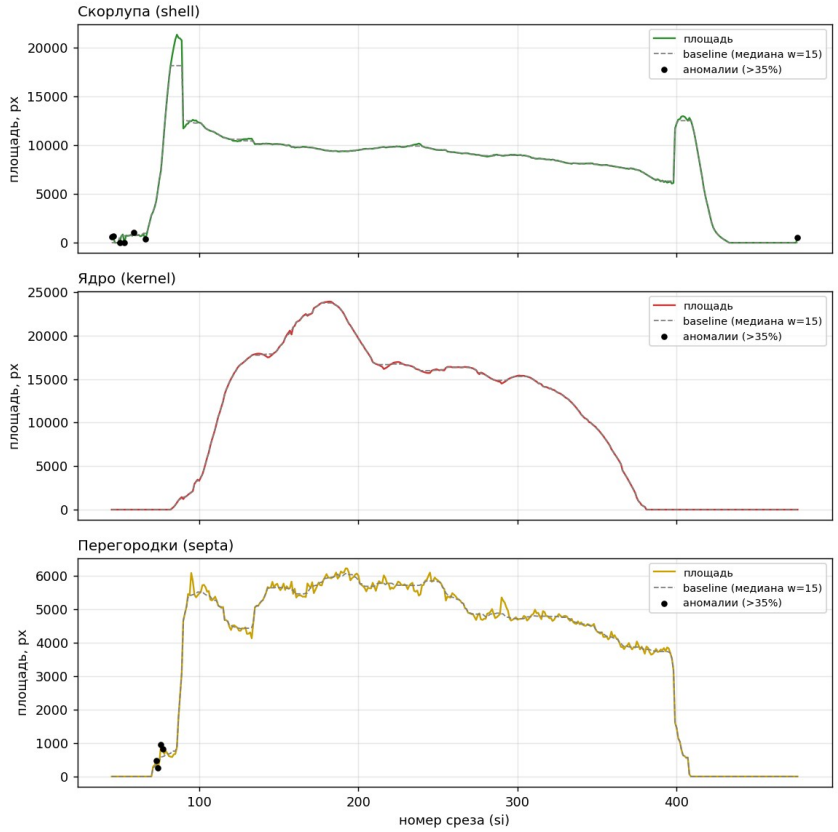


Рис. 30. Динамика площадей классов по номеру среза. Сплошная линия — площадь, пунктир — baseline (скользящая медиана, окно 15), чёрные точки — аномалии (отклонение > 35 %).

5. Построение 3D-мешей (build_mesh.py)

На вход второго этапа подаётся стопка overlay-изображений (результат сегментации). Скрипт build_mesh.py восстанавливает из них трёхмерный массив меток, строит для каждого класса полигональную поверхность методом Marching Cubes и сохраняет меши в OBJ и PLY. Ниже описаны метод и последовательная отладка артефактов в формате «метод → результат».

5.1. Декодирование overlay в метки (decode_labels)

Каждый пиксель overlay (в BGR) классифицируется по соотношению каналов, повторяющему цвета make_overlay: скорлупа — зелёный (G доминирует, R ≈ B), ядро — красный (R доминирует), перегородки — жёлтый (G ≈ R, оба велики, B мал).

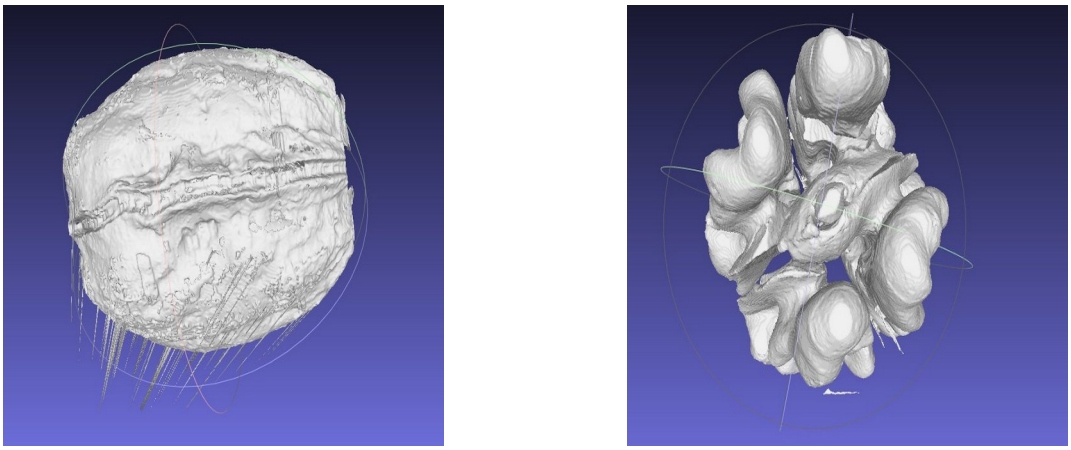
```
is_shell      = (g-r>35) & (g-b>35) & (abs(r-b)<40) & (g>40)
is_kernel     = (r-g>30) & (r-b>30) & (r>40)
is_partitions = (g>40)&(r>40)&(abs(g-r)<45)&(g-b>30) & ~is_shell & ~is_kernel
```

Срезы складываются в 3D-массив volume формы (Z, Y, X) с метками 0/1/2/3 (фон/скорлупа/ядро/перегородки).

5.2. Базовый метод и первые артефакты

Простейший пайплайн (Marching Cubes по каждому классу с лёгким сглаживанием) сразу выявил три типа артефактов. Они показаны на «сырых» результатах ниже.

Проблема 1 — «шипы вниз» у перегородок и ядра. Множество тонких острых выступов, направленных от модели (Рис. 31).



перегородки: множество шипов *ядро: шипы слабее*

Рис. 31. Артефакты базового метода: острые выступы у перегородок и ядра.

Проблема 2 — плавающие фрагменты скорлупы: отдельные куски поверхности «висят» вне основной оболочки (Рис. 32).

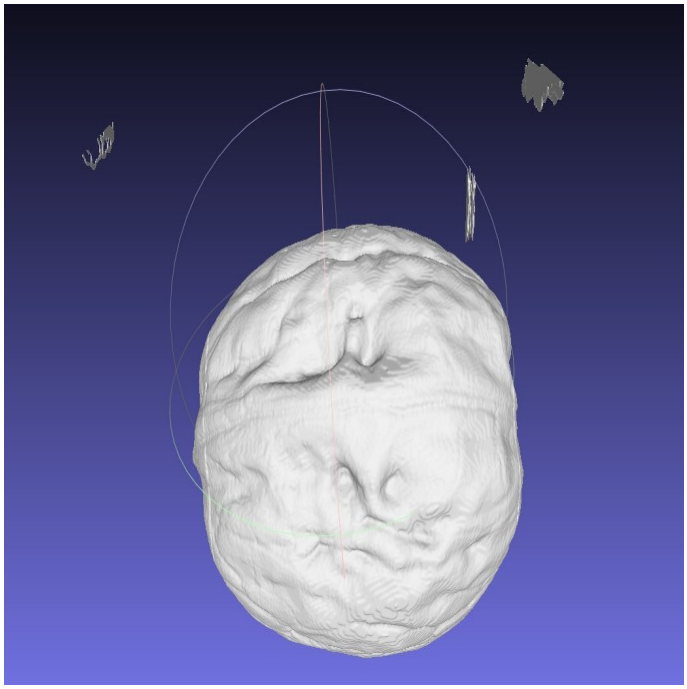


Рис. 32. Артефакт базового метода: плавающие фрагменты скорлупы вне основной оболочки.

5.3. Отладка: очистка и устранение артефактов

Причины артефактов были найдены прямым анализом объёма; для каждой реализован отдельный шаг очистки.

Шаг 1 — двухуровневая очистка компонент и обрезка краёв

Диагностика показала, что «шипы» порождаются мелкими изолированными пикселями на отдельных срезах: при Marching Cubes каждый такой кластер даёт острый выступ. Часть срезов скорлупы к тому же содержала пиксели только у нижнего края кадра (орех выходил за границу среза) — обрыв поверхности интерпретировался как открытый край и вытягивался в «шип». Реализованы:

— 2D-очистка каждого среза (`clean_slice_2d`): удаление компонент меньше MIN_COMPONENT_AREA = 100 пикселей и обнуление полосы EDGE_MARGIN = 5 пикселей вдоль краёв кадра;

— 3D-очистка (`remove_small_3d_components`): удаление изолированных пространственных компонент меньше MIN_3D_COMPONENT_VOXELS = 500 вокселей.

Для перегородок это сократило число 3D-компонент с 739 до 1 значимой; для скорлупы исчезли все плавающие фрагменты.

Шаг 2 — сохранение всех значимых долей (а не только наибольшей)

Первая версия очистки mesh оставляла только наибольшую связную компоненту. Для ядра это было ошибкой: ядро ореха состоит из нескольких раздельных лопастей, и все они нужны. После «жадной» очистки терялись целые доли (Рис. 33).

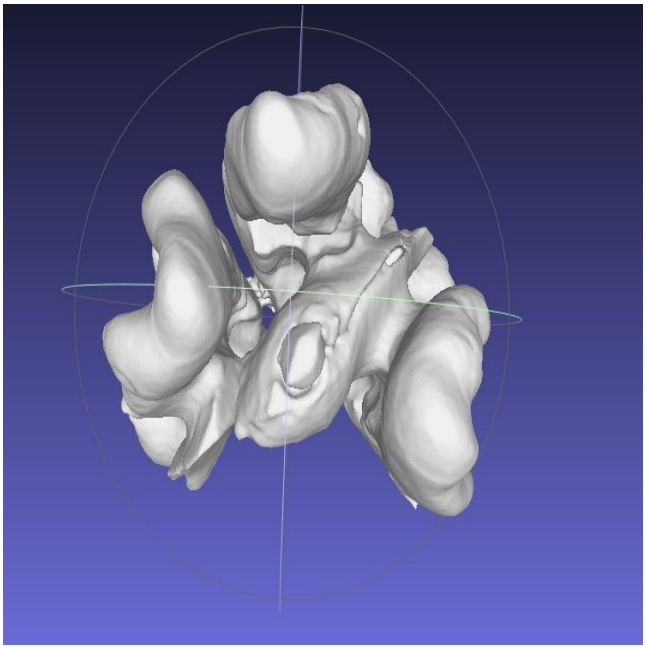


Рис. 33. Регрессия: при сохранении только наибольшей компоненты у ядра пропали целые доли.

Исправление: компонента удаляется только если её число граней меньше 1 % от наибольшей. Так отсекается лишь мусор, а все анатомические доли сохраняются.

```
components = mesh.split(only_watertight=False)
max_faces  = max(len(m.faces) for m in components)
threshold  = max(100, max_faces * 0.01)        # порог значимости
kept = [m for m in components if len(m.faces) >= threshold]
mesh = trimesh.util.concatenate(kept)
```

Шаг 3 — FloodFill только для скорлупы и ядра

На перегородках обнаружился паразитный тонкий «кольцевой» артефакт внутри модели (Рис. 34). Причина: операция заполнения дыр floodFill применялась ко всем классам. Но перегородки на ряде срезов (si ≈ 162–192) образуют почти замкнутый контур вокруг всего ореха; floodFill заполнял всё пространство внутри контура, превращая тонкие перегородки в сплошной диск, внутренняя поверхность которого и давала кольцо при Marching Cubes. На проблемных срезах заполнение добавляло до 45–48 тыс. пикселей — почти весь срез.

Исправление: floodFill применяется только к скорлупе и ядру (это сплошные объекты с реальными внутренними дырами). Для перегородок выполняется лишь лёгкое морфологическое закрытие ядром 5×5, без заполнения. Объём перегородок сократился почти вдвое (с ≈ 954 тыс. до ≈ 497 тыс. вокселей) — ровно за счёт артефактных данных.

```
is_solid = label in (LABEL_SHELL, LABEL_KERNEL)
sl = cv2.morphologyEx(sl, cv2.MORPH_CLOSE, kernel5)
if is_solid:                          # дыры заполняем только у сплошных классов
    cv2.floodFill(flooded, flood_mask, (0, 0), 1)
    sl = sl | (flooded == 0)
```

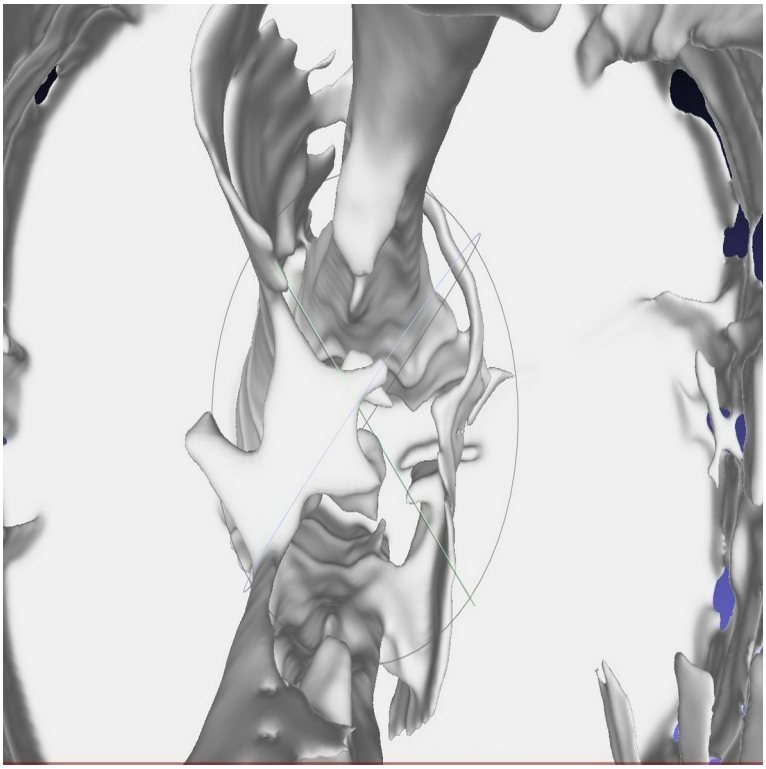


Рис. 34. Паразитное кольцо внутри перегородок (до исправления). Устранено отключением floodFill для класса перегородок.

Шаг 4 — low-poly режим (видимые полигоны)

По требованиям проекта (работа с сетками) поверхность не должна быть чрезмерно гладкой — полигоны должны быть различимы при максимальном приближении. Изначально применялись два сглаживающих шага: гауссово размытие объёма (sigma = 1.0) перед Marching Cubes и лапласово сглаживание меша (5 итераций). Оба отключены: размытие закомментировано, число итераций Лапласа по умолчанию выставлено в 0. В результате Marching Cubes выдаёт «ступенчатую» воксельную поверхность с чёткой полигональной структурой (Рис. 35).

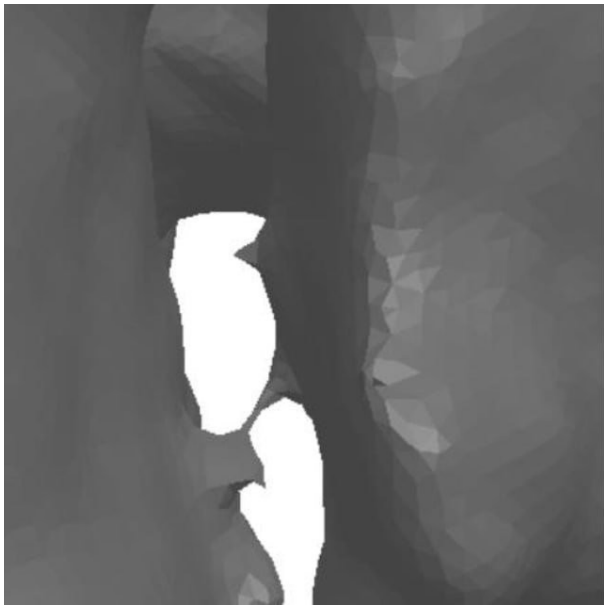


Рис. 35. Поверхность в режиме low-poly: отдельные полигоны различимы при увеличении.

Баланс между «сырым» и гладким видом регулируется параметром --smooth (число итераций Лапласа) и опциональным гауссовым размытием — оба вынесены так, чтобы их можно было включить одной строкой.

5.4. Итоговые меши

После всех шагов очистки построены три согласованных меша. Их размерности приведены в таблице ниже, визуализация — на Рис. 36–38.

Класс	Вершин	Треугольников
Скорлупа (shell)	269 620	539 236
Ядро (kernel)	302 432	605 100
Перегородки (septa)	638 372	1 296 584

Таблица 2. Размерность итоговых мешей (режим low-poly, шаг срезов = 3).



Рис. 36. Итоговый меш скорлупы.



Рис. 37. Итоговый меш ядра: раздельные лопасти сохранены.



Рис. 38. Итоговый меш перегородок (без паразитного кольца).

Параметры запуска build_mesh.py

Скрипт принимает диапазон срезов, шаг прореживания и параметры очистки. Типовые команды:

```
python build_mesh.py                          # все срезы из INPUT_DIR
python build_mesh.py 45 476 --step 3          # диапазон si 45..476, каждый 3-й
срез
python build_mesh.py --smooth 1               # лёгкое сглаживание (полигоны ещё
видны)
```

Ключевые константы вынесены в начало файла: MIN_COMPONENT_AREA, MIN_3D_COMPONENT_VOXELS, EDGE_MARGIN, VOXEL_SPACING. Параметр шага --step влияет на шаг по оси Z (учитывается в spacing для корректных пропорций модели).

6. Результаты

В ходе работы реализована полная цепочка от КТ-стопки до трёхмерных моделей классов ореха.

- Реализован и отлажен параметрический пайплайн сегментации segm.py с четырьмя стадиями (маска ореха → скорлупа/интерьер → ядро → перегородки). Система PARAM_ANCHORS с линейной интерполяцией обеспечивает плавную адаптацию ко всему диапазону срезов без правок логики.
- Трёхканальный детектор перегородок (Frangi + кольцо вокруг ядра + зона скорлупы) покрывает разные типы перемычек с разными механизмами контрастности.
- Расширенный debug-режим с нумерованными промежуточными изображениями делает диагностику параметров воспроизводимой и систематической; именно на нём построена пошаговая часть этого отчёта (48 кадров для среза 303).
- Качество подтверждено количественно: средний Dice 0.860 (ядро 0.929, скорлупа 0.885, перегородки 0.773) и гладкие кривые площадей по объёму.

— Реализован build_mesh.py: декодирование overlay в метки, Marching Cubes по классам, двухуровневая очистка (2D + 3D), обрезка краёв кадра, сохранение всех значимых долей ядра, раздельное заполнение дыр (floodFill только для сплошных классов) и low-poly режим с различимыми полигонами.

— Построены три согласованных меша (скорлупа, ядро, перегородки) в форматах OBJ/PLY, пригодные для дальнейшей работы с сетками.

7. Структура репозитория

Файлы и результаты организованы следующим образом:

```
etalon_masks/                  # эталонные маски + исходные TIFF
notes/                         # заметки, метрики, отчёты
prj.lab/
  __init__.py
  segm.py                      # пайплайн сегментации
  mesh_builder/
    build_mesh.py              # построение 3D-мешей
    input.rar                  # overlay-срезы (вход построения мешей)
  results/
    mesh_builder_results/      # shell/kernel/partitions .obj + .ply
    segmentation_results/      # overlay_si***.png (результат сегментации)
    volume_slicer_results/     # результаты нарезки объёма
  utilities/
    area_check.py              # контроль площадей по объёму
    evaluate.py                # подсчёт Dice/IoU
    recolor_mask.py            # ручная перекраска масок (перегородки/ядро)
    segment_from_masks.py      # сборка масок в overlay
  volume_slicer/
    __init__.py
    vs.py                      # нарезка/сборка объёма
.gitignore
```

Воспроизведение с нуля: (1) python prj.lab/segm.py — сегментация всех срезов в overlay; (2) python prj.lab/utilities/evaluate.py и area_check.py — контроль качества; (3) python prj.lab/mesh_builder/build_mesh.py 45 476 --step 3 — построение мешей. Промежуточные шаги любого среза доступны через python segm.py --mode debug.